FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 SPECIAL TOPIC IN DATA ENGINEERING

SECTION 01

APACHE SPARK TUTORIAL

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 5

Student Name	Matric No.
GOE JIE YING	A23CS0224
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187

1. Business Understanding

The Brazilian education institution lacks a method to efficiently evaluate systemic disparities, infrastructure gaps and administrative performance of the national school system. Critical data were collected over a decade and stored in the *Censo Escolar* datasets. However, these datasets in CSV format are extremely large and overloaded with highly dimensional data, making it hard to extract any valuable insights from it without a centralized, high performance analytical environment.

In this project, a simple data engineering and business intelligence pipeline that transforms this massive raw dataset into actionable strategic insights will be designed and implemented. This project aims to:

1. Optimize storage and performance by using Apache Parquet format
2. Design and populate a centralised data warehouse using Star Schema
3. Construct clear BI dashboards using Metabase

This end-to-end modernization will present a centralized single source of truth, allowing easier access to analyse and interpret the Censo Escolar dataset.

2. Data Understanding

The Brazilian National Basic Education Census (*Censo Escolar*) datasets provided by the National Institute for Educational Studies and Research Anísio Teixeira, contains microdata from an annual nationwide survey that covers all basic education institutions in Brazil. The schema consists of 370 variables per school record, capturing a highly multidimensional view of the national educational ecosystem. The columns can be roughly categorised into 6 logical domains:

1. Geographic and Administrative Demographics
2. Physical Infrastructure and Utilities
3. Facilities and Accessibility
4. Technology and Connectivity
5. Human Resources and Educational Material
6. Enrollments, Teachers and Class Groups (Metrics Matrix)

2.1 Data Types, Distribution and Data Quality

The table below shows three primary data types for the datasets.

Data Types	Example Columns	Data Description
Categorical	NO_REGIAO (region_name)	The name of the geopolitical region where the school is located
	SG_UF (state_abbreviation)	The two-letter state acronym representing the school's location
	TP_SITUACAO_FUNCIONAMENTO (operational_status)	A categorical code indicating if the school is currently active, paralyzed,

		extinct, or closed
Boolean	IN_AGUA_POTAVEL (potable_water)	The access to water that meets legal standards for drinking (0/1)
	IN_LABORATORIO_INFORMATICA (computer_lab)	The presence of operational computer lab for students (0/1)
	IN_INTERNET_ALUNOS (internet_for_students)	The presence of internet connection fro students access (0/1)
Numerical (Discreate)	QT_SALAS_EXISTENTES (total_existing_classrooms)	The total number of classrooms in school
	QT_COMP_ALUNO (total_student_computers)	The total count of computers for student use
	QT_FUNCIONARIOS (total_school_employees)	The total headcount of all employees working at the school

Table 2.1: Three Primary Data Type for the Datasets

Analysis on the dataset on the number of student enrollment has shown a right-skewed spatial distribution and a contracting temporal trend in school census data over 11 years (2010–2021). Total enrollment dropped steadily from a peak of 51.5M to 46.7M. In the year 2021 specifically, the geographic distribution of the number of enrollment shows that the data is highly concentrated in the Southeast, with the state of São Paulo acting as a massive outlier with over 9.8M enrollments.

A comprehensive data quality check is performed at the dataset:

1. Checking for Missing or NULL Values

```
SELECT
    COUNT(*) AS total_records,
    COUNT(qt_mat_bas) AS non_null_enrollments,
    SUM(CASE WHEN qt_mat_bas IS NULL THEN 1 ELSE 0 END) AS
null_enrollment_count,
    ROUND(100.0 * SUM(CASE WHEN qt_mat_bas IS NULL THEN 1 ELSE 0 END) /
COUNT(*), 2) || '%' AS null_percentage
FROM fact_censo_escolar;
```

Results:

	total_records bigint	non_null_enrollments bigint	null_enrollment_count bigint	null_percentage text
1	2792984	2792984	0	0.00%

0 null count and 0% null percentage indicates the data does not have null values.

2. Checking for Duplicated Records

```
SELECT
```

```
        "CO_UF",
        "CO_MUNICIPIO",
        COUNT(*) AS occurrence_count
    FROM dim_local
    GROUP BY "CO_UF", "CO_MUNICIPIO"
    HAVING COUNT(*) > 1;
```

Results:

	CO_UF text	CO_MUNICIPIO text	occurrence_count bigint

No rows returned indicates that the data does not have duplicate rows.

2.2 Relational Data Model

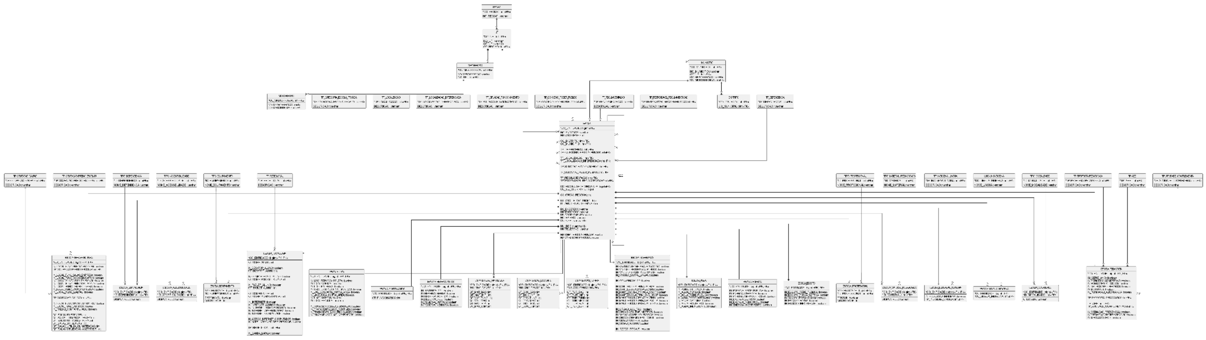

Figure 2.2: [Relational Data Model]() of *Censo Escolar* dataset

3. Data Preparation

3.1 Extract

The data was collected from the Brazilian National Basic Education Census for the years 2010–2021. The datasets were downloaded as ZIP files containing multiple related files. The ZIP files were extracted and only the CSV files were kept.

3.2 Transform

The CSV data was transformed into Parquet format using Apache Spark.

Parquet was used because it is a columnar storage format that improves query performance by reading only the required columns instead of scanning the entire dataset, as commonly occurs with CSV files.

Apache Spark was used to handle the large amount of data. Spark uses lazy evaluation where it does not load the data into memory until an action, such as write is called. In addition, Spark supports parallel data processing, which was explicitly configured with four shuffle partitions for the Spark session.

3.3 Load

Before the data is loaded into the database, the dimension tables were created and populated with data in the PostgreSQL database named censo_escolar using Apache Spark via JDBC. Spark first processed the parquet data as DataFrames in memory, including selecting the relevant columns, removing duplicate records to ensure unique dimension values, and assigning surrogate keys, before writing the results to PostgreSQL using .write.jdbc(). The fact table structure, including primary keys and foreign key constraints, was defined using psycopg2. After that, Spark loaded the data into the fact table via JDBC.

4. Modeling

4.1 Data Warehouse Architecture

The data warehouse follows a layered architecture, from data extraction, data transformation to loading stages (ETL). Raw data from the Brazilian National Basic Education Census is processed using Apache Spark, where it is cleaned and structured into Parquet format. The transformed data is then used to construct a star schema model with a central fact table and multiple dimension tables. The final structured data is loaded into a PostgreSQL database, named censo_escolar, which serves as the data warehouse layer. Metabase is used as the business intelligence layer for data visualization and analysis. Figure 4.1 shows the data warehouse architecture.

The architecture of the data warehouse is shown through the following layers:

- Data Source Layer: CSV files from census dataset (2010 - 2021)
- Processing Layer: Apache Spark (ETL)
- Storage Layer: PostgreSQL data warehouse
- Presentation Layer: Metabase dashboards

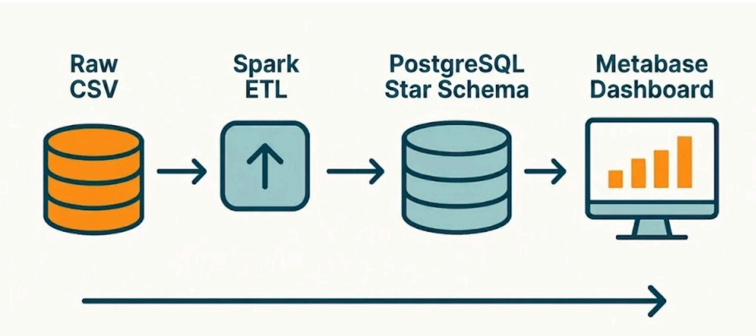

Figure 4.1: Data Warehouse Architecture

4.2 Star Schema Design

The data warehouse is designed using a star schema architecture, which consists of one central fact table surrounded by multiple dimension tables. The relationship of fact and dimensions are connected through surrogate key-based join operations. Figure 4.2 shows the star schema design.

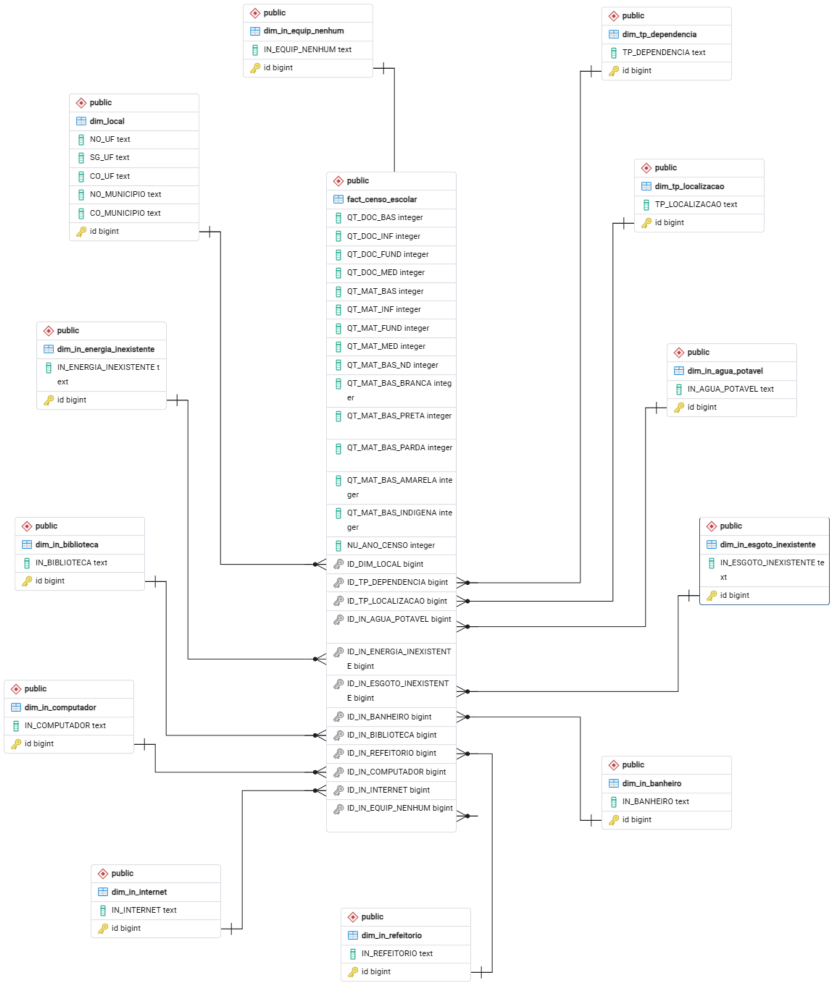

Figure 4.2: Star Schema Design

4.3 Fact Table

A central fact table named fact_censo_escolar (Educational Census Fact Table) was created, which stores quantitative measures derived from the Brazilian National Basic Education Census dataset. Each column in the fact table is explained in the section below.

4.3.1 Measures

- QT_DOC_BAS (Number of Basic Education Teachers)
- QT_DOC_INF (Number of Early Childhood Education Teachers)
- QT_DOC_FUND (Number of Primary Education Teachers)
- QT_DOC_MED (Number of Secondary Education Teachers)
- QT_MAT_BAS (Number of Basic Education Enrollments)
- QT_MAT_INF (Number of Early Childhood Education Enrollments)
- QT_MAT_FUND (Number of Primary Education Enrollments)

- QT_MAT_MED (Number of Secondary Education Enrollments)
- QT_MAT_BAS_ND (Number of Enrollments with No Declared Ethnicity)
- QT_MAT_BAS_BRANCA (Number of White Student Enrollments)
- QT_MAT_BAS_PRETA (Number of Black Student Enrollments)
- QT_MAT_BAS_PARDA (Number of Mixed-Race Student Enrollments)
- QT_MAT_BAS_AMARELA (Number of Asian Student Enrollments)

4.3.2 Time Attribute

- NU_ANO_CENSO (Census Year)

4.3.3 Foreign Keys

Foreign keys were used to connect the fact table with dimension tables.

- ID_DIM_LOCAL (Location Dimension ID)
- ID_TP_DEPENDENCIA (School Dependency Type ID)
- ID_TP_LOCALIZACAO (School Location Type ID)
- ID_IN_AGUA_POTAVEL (Potable Water Availability ID)
- ID_IN_ENERGIA_INEXISTENTE (No Electricity Availability ID)
- ID_IN_ESGOTO_INEXISTENTE (No Sewage Availability ID)
- ID_IN_BANHEIRO (Bathroom Availability ID)
- ID_IN_BIBLIOTECA (Library Availability ID)
- ID_IN_REFEITORIO (Cafeteria Availability ID)
- ID_IN_COMPUTADOR (Computer Availability ID)
- ID_IN_INTERNET (Internet Availability ID)
- ID_IN_EQUIP_NENHUM (No Equipment Availability ID)

4.4 Dimension Table

Twelve dimension tables were created and populated using the transformed Parquet data. Duplicate records were removed before loading and each dimension record was assigned an id as the primary key of the table.

The dimension tables include:

- dim_local (Location Dimension)
- dim_tp_dependencia (School Dependency Type Dimension)
- dim_tp_localizacao (School Location Type Dimension)
- dim_in_agua_potavel (Potable Water Dimension)
- dim_in_energia_inexistente (No Electricity Dimension)
- dim_in_esgoto_inexistente (No Sewage Dimension)
- dim_in_banheiro (Bathroom Dimension)
- dim_in_biblioteca (Library Dimension)
- dim_in_refeitorio (Cafeteria / Dining Hall Dimension)
- dim_in_computador (Computer Dimension)
- dim_in_internet (Internet Dimension)
- dim_in_equip_nenhum (No Equipment Dimension)

4.5 Relationship Model

During the ETL process implemented in Apache Spark, the fact table was created with foreign key constraints referencing dimension tables. The table was initially populated using the original dimension attributes and fact measurement columns from the Parquet file. Then, it was joined with the corresponding dimension tables to replace the original attribute values with their respective dimension table IDs.

5. Evaluation

5.1 System Performance and Data Validation

The data warehouse and ETL pipeline successfully processed more than 5 million educational census records by transforming the raw dataset into a structured star schema using Apache Spark. Additionally, the star schema was loaded and stored in PostgreSQL. This execution of Spark job shows the system's ability to support large-volume analytical data processing.

After the loading process is complete, a SQL COUNT query was executed on the fact table to verify the records stored and a PostgreSQL meta-command (\d) was used to inspect table structure to check if the fact table was correctly linked to its corresponding dimension tables through foreign key constraints. The result for each query is shown as below:

```
censo_escolar=# SELECT COUNT(*)
FROM fact_censo_escolar;
  count
---------
 5143688
(1 row)
```

Figure 5.1: Result of execution using COUNT

```
                  Table "public.fact_censo_escolar"
          Column           |  Type   | Collation | Nullable | Default
---------------------------+---------+-----------+----------+---------
 QT_DOC_BAS                | integer |           |          |
 QT_DOC_INF                | integer |           |          |
 QT_DOC_FUND               | integer |           |          |
 QT_DOC_MED                | integer |           |          |
 QT_MAT_BAS                | integer |           |          |
 QT_MAT_INF                | integer |           |          |
 QT_MAT_FUND               | integer |           |          |
 QT_MAT_MED                | integer |           |          |
 QT_MAT_BAS_ND             | integer |           |          |
 QT_MAT_BAS_BRANCA         | integer |           |          |
 QT_MAT_BAS_PRETA          | integer |           |          |
 QT_MAT_BAS_PARDA          | integer |           |          |
 QT_MAT_BAS_AMARELA        | integer |           |          |
 QT_MAT_BAS_INDIGENA       | integer |           |          |
 NU_ANO_CENSO              | integer |           |          |
 ID_DIM_LOCAL              | bigint  |           |          |
 ID_TP_DEPENDENCIA         | bigint  |           |          |
 ID_TP_LOCALIZACAO         | bigint  |           |          |
 ID_IN_AGUA_POTAVEL        | bigint  |           |          |
 ID_IN_ENERGIA_INEXISTENTE | bigint  |           |          |
 ID_IN_ESGOTO_INEXISTENTE  | bigint  |           |          |
 ID_IN_BANHEIRO            | bigint  |           |          |
 ID_IN_BIBLIOTECA          | bigint  |           |          |
 ID_IN_REFEITORIO          | bigint  |           |          |
 ID_IN_COMPUTADOR          | bigint  |           |          |
 ID_IN_INTERNET            | bigint  |           |          |
 ID_IN_EQUIP_NENHUM        | bigint  |           |          |
Foreign-key constraints:
    "fk_agua" FOREIGN KEY ("ID_IN_AGUA_POTAVEL") REFERENCES dim_in_agua_potavel(id)
    "fk_banheiro" FOREIGN KEY ("ID_IN_BANHEIRO") REFERENCES dim_in_banheiro(id)
    "fk_biblioteca" FOREIGN KEY ("ID_IN_BIBLIOTECA") REFERENCES dim_in_biblioteca(id)
    "fk_computador" FOREIGN KEY ("ID_IN_COMPUTADOR") REFERENCES dim_in_computador(id)
    "fk_energia" FOREIGN KEY ("ID_IN_ENERGIA_INEXISTENTE") REFERENCES dim_in_energia_inexistente(id)
    "fk_equip" FOREIGN KEY ("ID_IN_EQUIP_NENHUM") REFERENCES dim_in_equip_nenhum(id)
    "fk_esgoto" FOREIGN KEY ("ID_IN_ESGOTO_INEXISTENTE") REFERENCES dim_in_esgoto_inexistente(id)
    "fk_internet" FOREIGN KEY ("ID_IN_INTERNET") REFERENCES dim_in_internet(id)
    "fk_local" FOREIGN KEY ("ID_DIM_LOCAL") REFERENCES dim_local(id)
    "fk_refeitorio" FOREIGN KEY ("ID_IN_REFEITORIO") REFERENCES dim_in_refeitorio(id)
    "fk_tp_dependencia" FOREIGN KEY ("ID_TP_DEPENDENCIA") REFERENCES dim_tp_dependencia(id)
    "fk_tp_localizacao" FOREIGN KEY ("ID_TP_LOCALIZACAO") REFERENCES dim_tp_localizacao(id)
```

Figure 5.2: Structure of the Fact table

The query confirmed that the data integration and storage were successful as the fact table contains more than 5 million records and correctly linked to all dimension tables.

5.2 Dashboard and Business Intelligence Evaluation

The PostgreSQL data warehouse was then integrated with Metabase to support dashboard creation and business intelligence analysis. With predefined relationships in the star schema, the dashboards can retrieve data efficiently. The analysis of educational patterns and infrastructure conditions can be performed in the dashboard using interactive filtering features, which supports data-driven decision-making.

The following dashboard was developed for this case study.

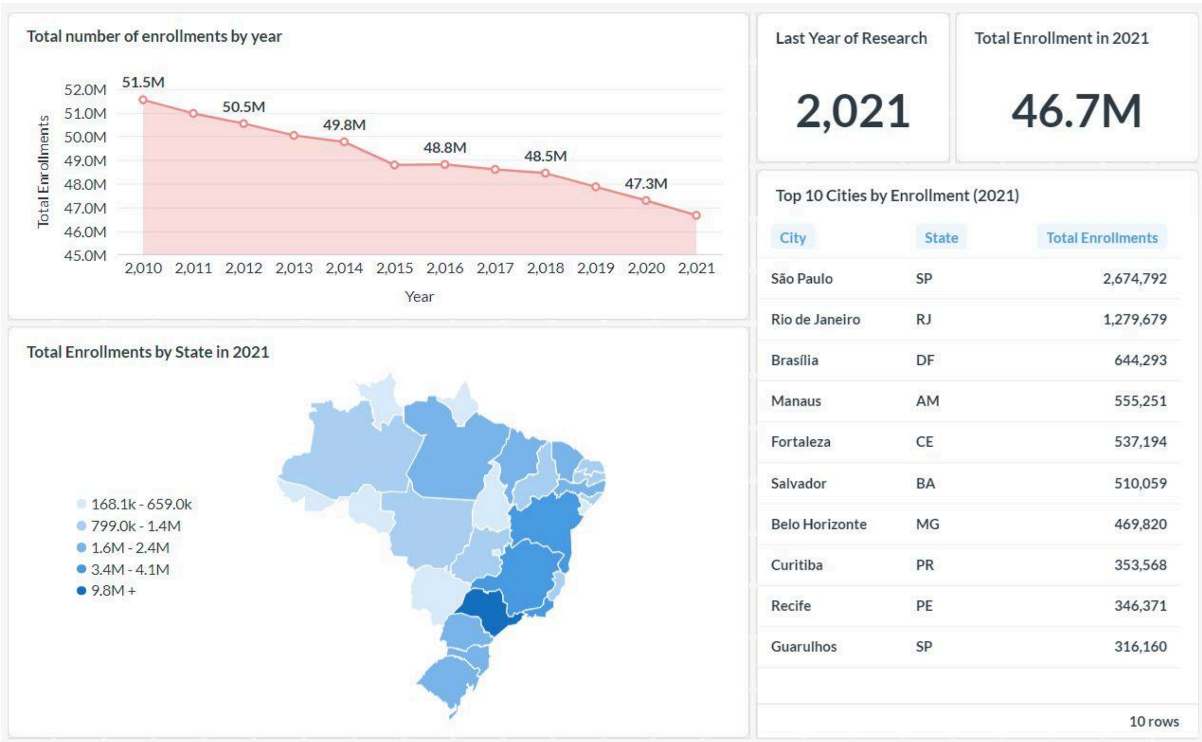

Figure 5.3: Dashboard Developed Using Metabase for Educational Data Analysis

5.3 Limitations and Future Improvements

Despite successful implementation, the ETL process is limited to batch processing only and does not support real time analytics. The data is processed in a fixed stage using Apache Spark so if new data is added to the pipeline, it would be reprocessed and reloaded into the system. Besides, the processes were executed manually through several scripts in this case to transform data in parquet and populate them into star schema. This can increase the risk of human error especially during joint operations.

The project can be further improved by applying streaming frameworks such as Apache Kafka to support realtime data ingestion and processing. Tools such as Apache Airflow can be utilized to automate the ETL pipeline, thereby reduce human intervention and better monitor the data workflow.

6. Reflection

Lam Yoke Yu

This tutorial provided us with hands-on experience to build an ETL pipeline that takes raw Brazilian school census data and structures it into a Star Schema in PostgreSQL, by using Apache Spark for handling large data.

One of the main challenges encountered during the implementation was when using Spark to read the data. Initially, the code was executed within a Python notebook environment, but the program ran for an exceptionally long time without producing any output. The issue was later identified by executing the program as a Python script, which enabled error messages to be displayed in the terminal. The problem was due to incompatible and missing dependencies. Additionally, the long execution time occurred because the process did not terminate properly after the error had occurred.

By analysing the reference implementation, a better understanding of the data transformation and loading process into PostgreSQL was achieved. Overall, this tutorial provided valuable exposure to Apache Spark and ETL pipeline development.

Goe Jie Ying

The development of an end-to-end data pipeline project using Apache Spark provides a valuable practical experience. Throughout the tutorial, the understanding of ETL concepts and dimensional data modeling becomes deeper and more meaningful. The importance of integrating them together became realistic and evident when a star schema was created successfully.

One of the challenges faced was defining and managing multiple dimension tables manually within the codes. Due to the large number of dimensions included, the key in some dimension tables was not connected properly with the fact table. As a result, while inspecting the star schema generated using pgAdmin, several dimension tables were disconnected with the fact table. To address this, the foreign key relationships were reviewed and configured manually using SQL statements. Finally, a well-structured star schema form.

A lesson learned is that defining the relationship between fact and dimension tables clearly using correct referencing keys can smoothen the process of data processing and modeling. It will make the process of performing further analysis effective, for instance, this helps to create a dashboard faster to discover pattern and insight.

Tan Yi Ya

An end-to-end data pipeline had been successfully developed with Apache Spark that transformed a massive and seemingly impossible to analyse dataset from CSV into Parquet format. The dataset is then further altered into centralised Star Schema, and visualised with metabase supported by docker, turning it into insights for analysis.

The main challenges faced during the implementation were mainly caused by operating system discrepancies and environmental constraints. The pipeline designed for a Linux ecosystem required some alteration to run in Windows environment. Additional difficulties were experienced during the integration between Apache Spark, PostgreSQL, Docker and Metabase. Configuration issues such as incorrect environment variables, port conflicts and database connection errors occasionally prevented services from communicating properly. Several troubleshooting steps were required to ensure that the containers were correctly networked and that Spark was able to write transformed data into PostgreSQL successfully.

Overall, this tutorial provided valuable practical experience in developing a complete big data pipeline using Apache Spark, PostgreSQL, Docker and Metabase. It also strengthened understanding of ETL workflows, data warehousing concepts, schema design and large-scale data processing for analytical purposes.